Observabilidad y versionado de prompts

MMIA 6013 — IA Generativa y Agentes · Sesión 17 · martes 6 de octubre de 2026

Semana 4 · RA-5 · ayer quedó una pregunta abierta: ¿dónde ocurren esos fallos?

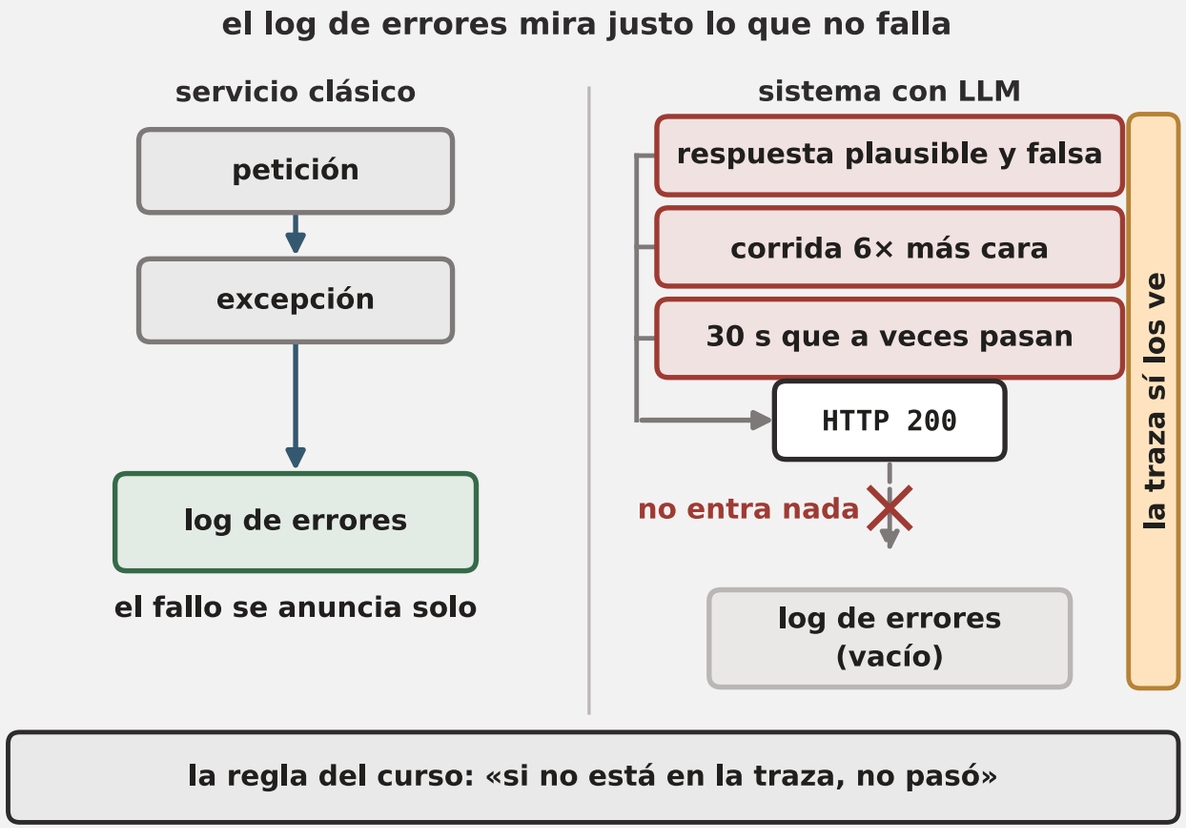
1 · el problema · 2 · cómo se traza y cómo se llama cada objeto · 3 · ejemplos · 4 · Langfuse: código, demo y cómo se ve

45 min de deck · 15 de «encuentra al culpable» · 10 de demo · 40 de cuaderno

Daniel Andrés Riofrío Almeida

1 · El problema: el fallo característico devuelve HTTP 200

- «Respuestas plausibles pero incorrectas, costos que se disparan sin errores visibles»
- «Latencias de 30 s que a veces pasan». El log de errores no ve ninguna de las tres
- En un servicio clásico el fallo lanza. Aquí la corrida que cuesta 6× termina bien
- «Los prompts son código» y «el costo es por token»: dos de las cinco diferencias con MLOps
- Observabilidad **no es logging**: es poder **reconstruir y atribuir** una corrida — qué pasó, cuánto costó, cuánto tardó, con qué modelo y con qué versión del prompt
- La regla del curso: «si no está en la traza, no pasó». Sin traza no se depura un fallo, no se atribuye un costo ni se demuestra una mejora

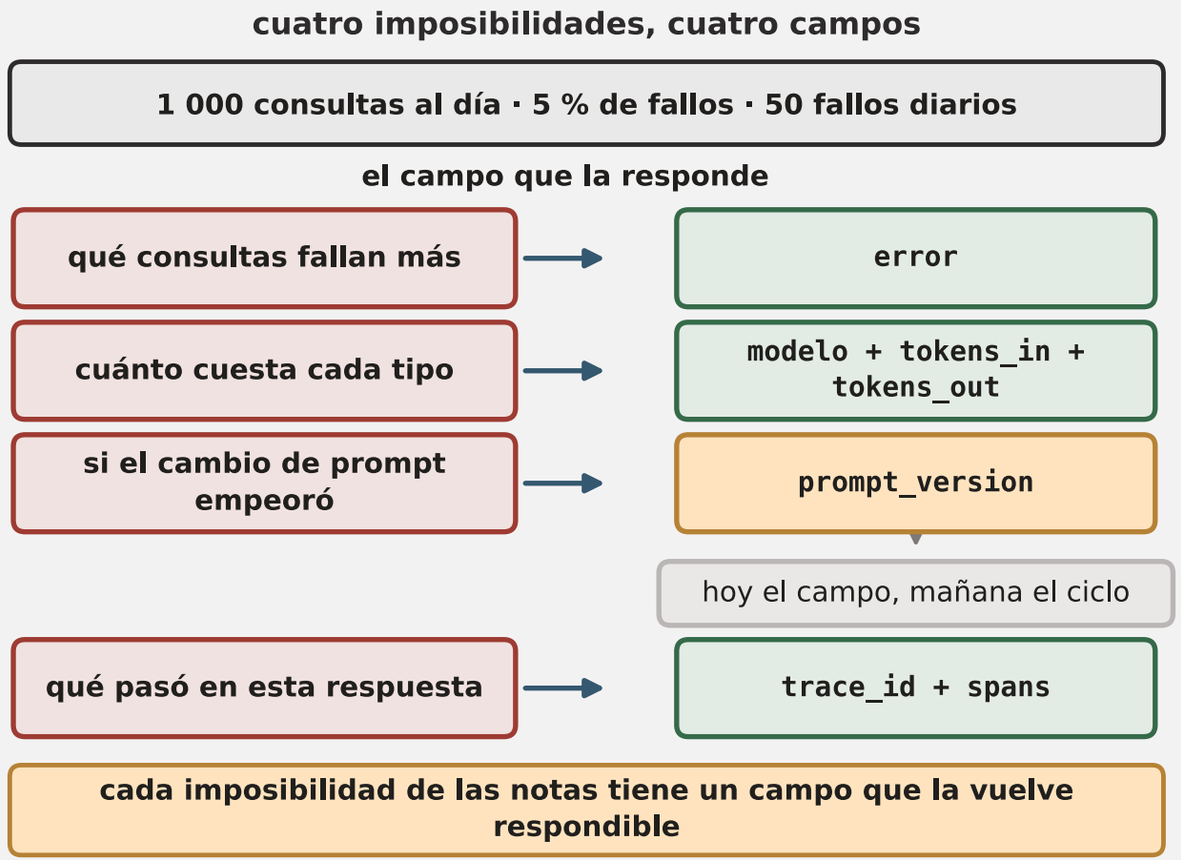


las tres formas de fallo son literales de curso/actividades/semana-4/s4-mar-explicacion.html, líneas 37–38

Fuente: curso/actividades/semana-4/s4-mar-explicacion.html, líneas 37–42 (las tres formas de fallo y la regla del curso, literales); fuentes/notas-teoricas/s4-llmops-y-finetuning.md §1, líneas 10–14 (las cinco diferencias entre LLMOps y MLOps); proceso/contexto/glosario.md, entrada «Observability» (la definición: reconstruir y atribuir, y que no es logging). Que el modo de fallo característico devuelva HTTP 200 es [AGREGADO - formulación de contenido.md §2.3]

1 · Por qué es un problema: cuatro preguntas sin respuesta

- 1.000 consultas al día y un 5 % de fallos: 50 fallos diarios
- Dos palabras que la diapositiva siguiente define: **traza** = una corrida entera; **span** = un paso dentro de ella
- Qué tipo de consulta falla más → el campo error del span
- Cuánto cuesta cada tipo → modelo, tokens_in, tokens_out
- Si un cambio de prompt empeoró algo → prompt_version
- Auditar una respuesta concreta → trace_id, e input y salida de cada span



las cuatro imposibilidades y la cuenta de los 50 fallos son literales de fuentes/notas-teoricas/s4-llmops-y-finetuning.md §3.1, líneas 103–107

Fuente: fuentes/notas-teoricas/s4-llmops-y-finetuning.md §3.1, líneas 103–107 (la cuenta de los 50 fallos diarios y las cuatro imposibilidades, literales). El mapeo de cada imposibilidad a su campo es [AGREGADO - formulación de contenido.md §2.1]

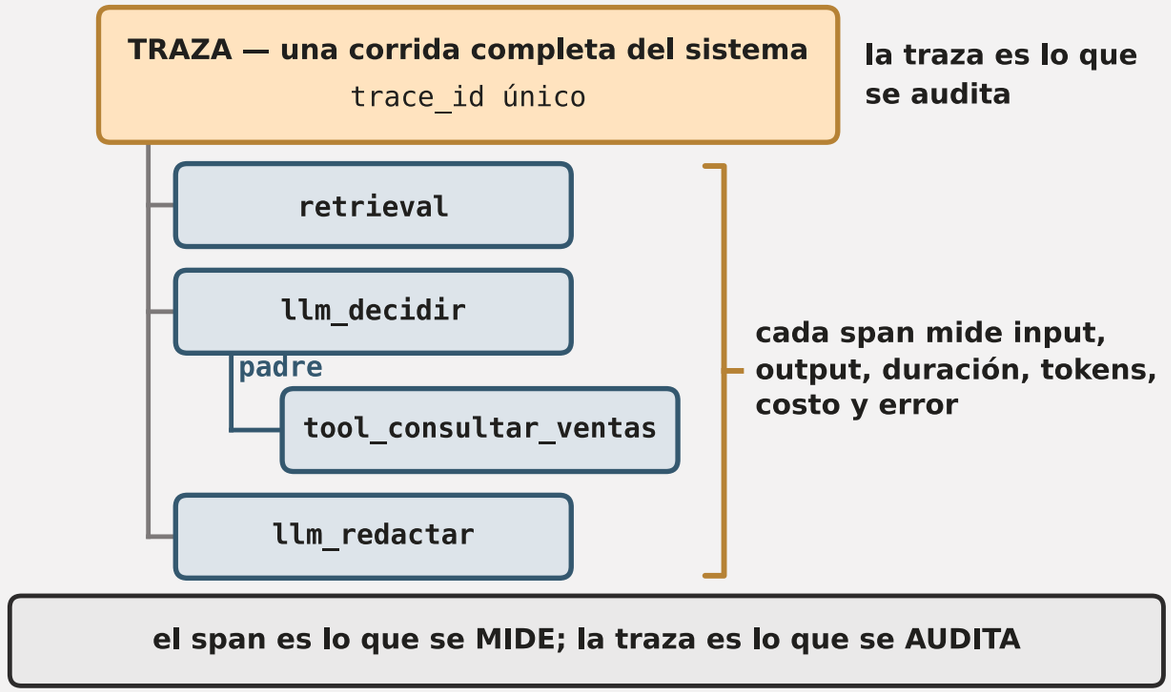
2 · Cómo se traza: traza, span y sus campos

17

La traza se audita; el span se mide

- «Una traza es el registro completo de una ejecución» y «tiene una ID única»
- «Contiene múltiples spans: llamadas al LLM, llamadas a tools, pasos del pipeline»
- «Cada span registra: input, output, duración, costo (tokens), errores»
- Los spans se anidan por un campo `padre`: el padre de un span es el span dentro del cual se abrió
- El span es lo que se MIDE; la traza es lo que se AUDITA
- No es la traza del agente de la 11: aquélla es una lista en memoria, sin tiempos ni costo

una traza, cuatro spans, un campo de parentesco



en la corrida que trae el repositorio los 40 spans tienen padre = null: la anidación está implementada y no se ejercita

Fuente: fuentes/notas-teoricas/s4-llmops-y-finetuning.md §3.2, líneas 111–114 (las tres viñetas, literales); curso/actividades/semana-4/s4-mar-explicacion.html, líneas 46–58 (la anidación y el campo padre); proceso/contexto/glosario.md, entradas «Trace / span» y «Agent trace». La distinción medir/auditar es [AGREGADO - contenido.md §3.1]

Ocho requisitos mínimos: once campos en cuatro familias

- Los **ocho**, en el orden del material: **1** identidad · **2** tiempos · **3** modelo · **4** tokens de entrada · **5** tokens de salida · **6** costo estimado · **7** versión del prompt · **8** error
- Dos requisitos son varios campos: **1** = trace_id, span_id, padre; **2** = inicio, duración. Por eso **8 requisitos son 11 campos**
- Cuatro familias: identidad **1** · tiempo **2** · economía **3–6**, **solo en el span del LLM** · cambio **7**, `prompt_version`, **una vez por traza**. El error **8** cruza las cuatro
- «Sin tokens y modelo no puedes atribuir costo; sin versión de prompt no puedes comparar cambios»
- La Parte 1 del Taller 4 (25 %) pide ocho de estos once campos: no pide span_id, padre ni inicio

quitar una familia apaga una pregunta entera

8 requisitos de la lista · 11 campos escritos · 4 familias

IDENTIDAD	TIEMPO	ECONOMÍA	CAMBIO
trace_id *span_id* *padre*	*inicio* **duración**	**modelo** **tokens_in** **tokens_out** **costo**	**prompt_version**
el trace_id en la traza, los otros dos en cada span	*en cada span*	*solo en los spans de llamada al LLM*	*una vez por traza*
sin ella no se puede juntar la corrida	sin ella «a veces tarda» sigue sin dato	sin ella el gasto total no se puede repartir	sin ella ninguna mejora es demostrable

y error, si lo hubo — atraviesa las cuatro familias

la Parte 1 del Taller 4 (25 %) pide 8 de estos 11 — span_id, padre e inicio no

la lista, literal: s4-mar-explicacion.html, línea 109 · lo que califica: taller-04-llmops.md, Parte 1, «Instrucciones», y resumir_trazas.py

Fuente: curso/actividades/semana-4/s4-mar-explicacion.html, línea 109 (la lista de los ocho requisitos y su cierre, literales; es la única lista completa del repositorio); curso/actividades/_fuentes/s4-mar.py, líneas 142 y 146–166 (en qué nivel escribe cada campo la implementación de referencia); curso/talleres/taller-04-llmops.md, Parte 1, «Instrucciones», puntos 1–4, y «Criterios de evaluación» (la Parte 1 y su 25 %); curso/labs/Lab-04-LLMOps/resumir_trazas.py, docstring (la estructura mínima: un campo traza compartido, sin span_id ni padre). La agrupación en cuatro familias es [AGREGADO - contenido.md §3.2]; el recuento de ocho de once, [AGREGADO - CÁLCULO DERIVADO]

¿Cuánto costó esta traza?

La traza de ejemplo del material: 4 spans, 2 165 ms, 2 800 tokens de entrada y 295 de salida

Y al pie, impreso: «total: 2165 ms · $0,011 · 4 spans»

A. $0,011, que es lo que dice la figura

B. $0,004275, que es lo que dan los precios de la tabla semestral

*C. No se puede calcular con lo que la figura muestra

Los milisegundos sí cuadran: 45 + 900 + 120 + 1 100 = 2 165

la traza de ejemplo del material, tal como la imprime

```
TRAZA
├─ retrieval                          45 ms
├─ llm_decidir     900 ms · in 1 200 out 85
├─ tool:consultar_sql                120 ms
├─ llm_decidir  1 100 ms · in 1 600 out 210
```

total 2 165 ms
cuadra: 45 + 900 + 120 + 1 100

total $0,011
de la línea de total

cómo se pone precio a unos tokens

un precio por millón de tokens de entrada y otro por millón de tokens de salida

y la tabla semestral trae un par de precios por fila

¿cuánto costó esta traza? A, B o C — un minuto en grupos

traza de ejemplo de curso/actividades/semana-4/s4-mar-explicacion.html, líneas 46–53 · precios de modelos-2026-1.json, verificada 2026-07-10

Fuente: curso/actividades/semana-4/s4-mar-explicacion.html, líneas 46–53 (la traza de ejemplo y su línea de total, literales); fuentes/modelos/modelos-2026-1.json, fila juez_economico — $1,00 de entrada y $5,00 de salida por 1M, verificada 2026-07-10. Las cuatro cuentas son [AGREGADO - CÁLCULO DERIVADO]; el desajuste está registrado como discrepancia 0.az

3 · Ejemplos: diez corridas y cuarenta spans del curso

17

272, 346 y 740 ms: ¿cuál publica el dashboard?

Las diez corridas de hoy dan tres números de latencia: 272, 346 y 740 ms

Uno es el p50, otro es la media y otro es el p95. ¿Cuál es cuál?

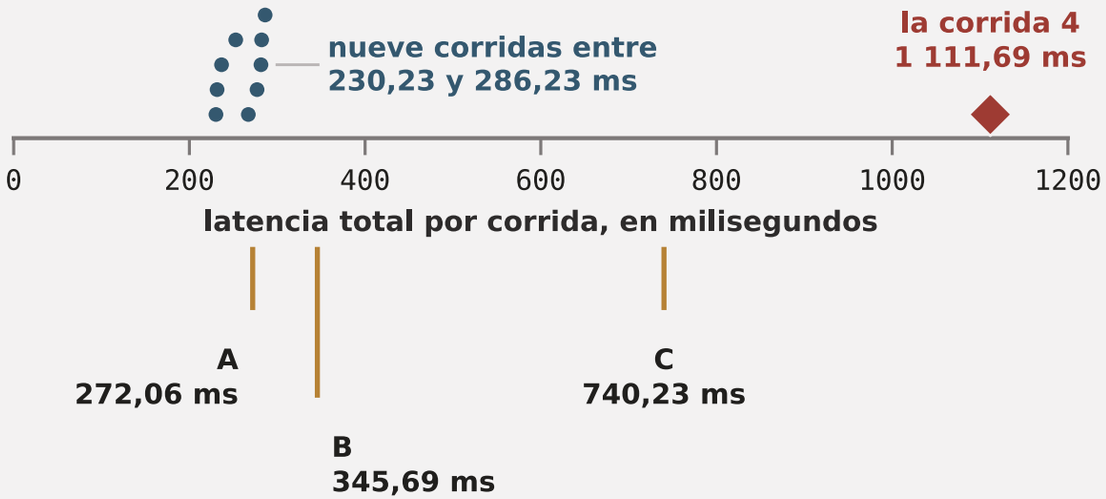
Entre las diez hay una corrida de 1 111,69 ms. ¿Cuál de los tres la delata?

Y la del viernes 17: ¿cuál de los tres dejó de pedir el Taller 4 el 2026-10-02, y por qué?

notas-s4 §6.1 monitorea «Latencia p95», con umbral «>3× latencia normal»

El Taller 4 pedía, hasta el 2026-10-01, «latencia promedio por ejecución»

las diez latencias de la corrida del curso



uno es el p50, otro la media y otro el p95 — ¿cuál es cuál, y cuál publican el sábado?

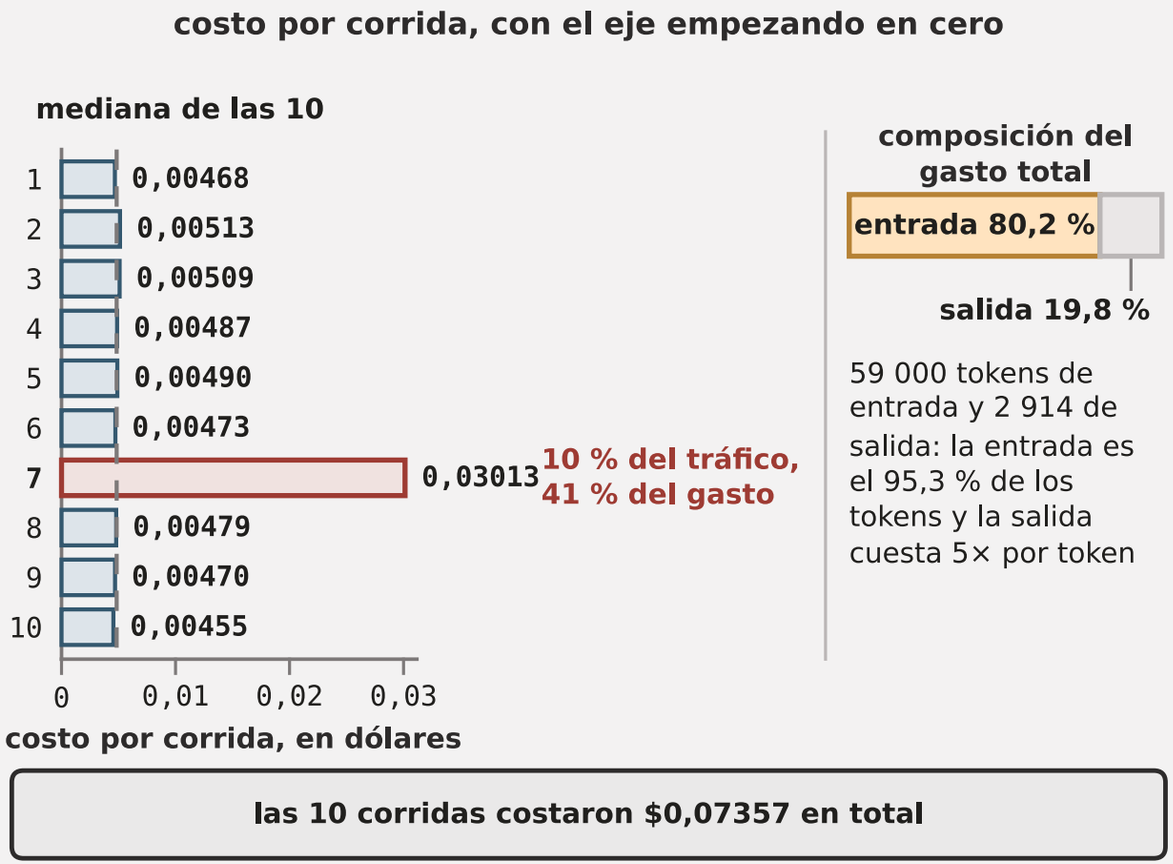
las notas de esta semana monitorean «Latencia p95», con umbral «> 3× latencia normal»	el enunciado del Taller 4 pedía «latencia promedio por ejecución» hasta el 2026-10-01

las diez latencias salen de curso/actividades/_fuentes/traces.jsonl; percentil lineal, el de np.percentile — _fuentes/s4-mar.py, líneas 197–198

Fuente: curso/actividades/_fuentes/traces.jsonl (los diez ms_total; percentil lineal, el de np.percentile, que es lo que manda graficar _fuentes/s4-mar.py, líneas 197–198); fuentes/notas-teoricas/s4-llmops-y-finetuning.md §6.1, línea 281; curso/talleres/taller-04-llmops.md en su versión del 2026-09-09 (git e394f5c), Parte 1, punto 3, y en la de hoy, Parte 1, «Entregables». Los tres estadísticos son [AGREGADO - CÁLCULO DERIVADO]

El 10 % del tráfico y el 41 % del gasto

- Las diez corridas costaron $0,07357 en total
- La corrida 7 sola costó $0,03013: el 41 % del gasto del día
- Es 6,2× la mediana de las diez, que está en $0,004835
- Los tokens de ENTRADA son el 80,2 % del gasto, aunque la salida cueste 5× por token
- Porque son el 95,3 % de los tokens: 59 000 de entrada frente a 2 914 de salida
- Precios: fuentes/modelos/modelos-2026-1.json, fila juez_economico, verificada 2026-07-10

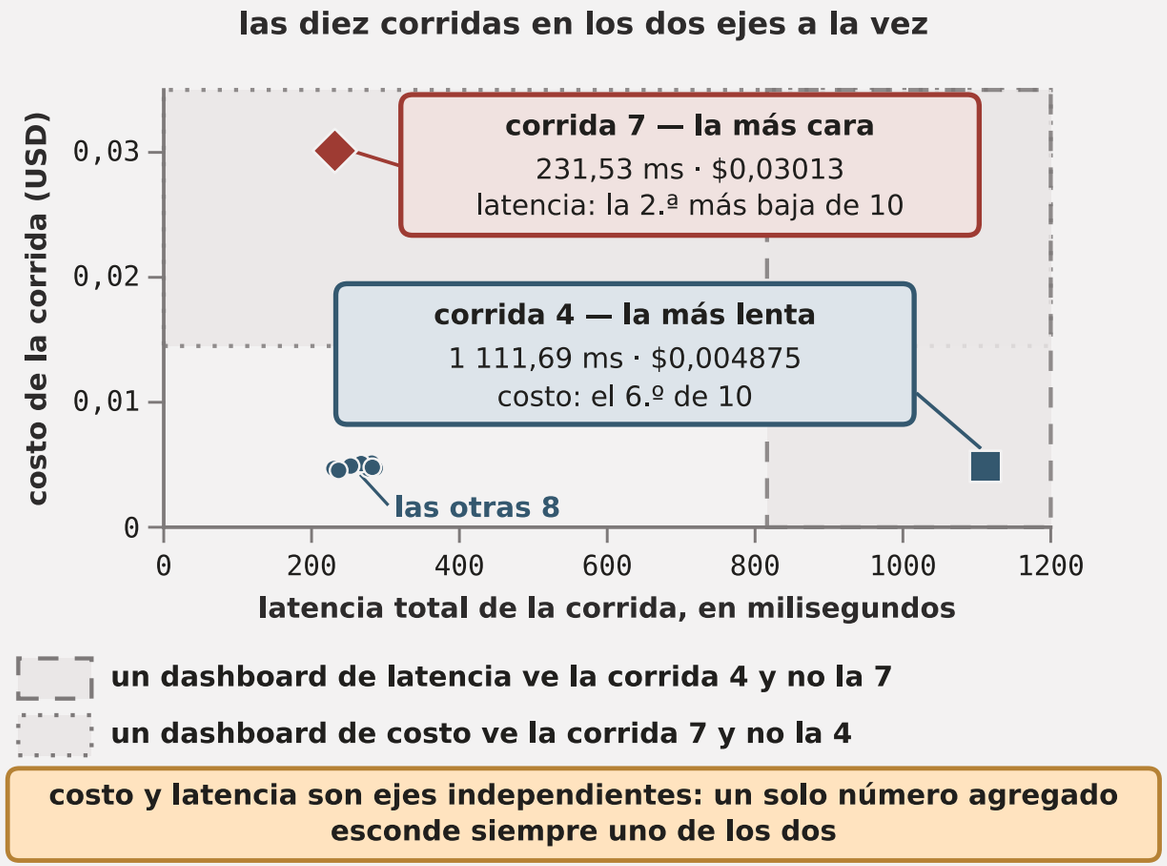


costos y tokens de traces.jsonl · precios de modelos-2026-1.json, fila juez_economico, verificada 2026-07-10

Fuente: curso/actividades/_fuentes/traces.jsonl (los diez costo_usd; el desglose por span de la corrida 7 se trabaja en la actividad, no aquí); fuentes/modelos/modelos-2026-1.json, fila juez_economico, $1,00/$5,00 por 1M, verificada 2026-07-10. Los porcentajes son [AGREGADO - CÁLCULO DERIVADO], comprobados contra los tokens

La corrida más cara es la segunda más rápida

- Corrida 4: 1 111,69 ms, la peor de las diez. Y su costo es el 6.º de diez: por debajo de la media, no de la mediana
- Corrida 7: 231,53 ms, la segunda mejor de las diez. Y costó $0,03013, 6,2× la mediana
- El dashboard de costo no ve la 4. El dashboard de latencia no ve la 7
- Costo y latencia son ejes independientes
- Un solo número agregado esconde siempre uno de los dos



costo y latencia son ejes independientes: un solo número agregado esconde siempre uno de los dos

traces.jsonl; las zonas marcan 3× la mediana de su eje · precios de modelos-2026-1.json, fila juez_economico, verificada 2026-07-10

Fuente: curso/actividades/_fuentes/traces.jsonl, trazas 4 y 7 (ms_total 1111,69 y 231,53; costo_usd 0,004875 y 0,03013, leídos del archivo); precios de fuentes/modelos/modelos-2026-1.json, fila juez_economico, verificada 2026-07-10. La lectura de los dos ejes como independientes es [AGREGADO - contenido.md §4.4]

Encuentra al culpable - 15 minutos

- Por grupos, sobre las trazas 4 y 7 completas: cuatro spans cada una, con sus atributos
- 1 · ¿Qué span explica el costo de la 7? 2 · ¿Qué span explica la latencia de la 4?
- 3 · Si solo pueden arreglar una esta semana, ¿cuál, y con qué argumento?
- El material pide una tercera traza, «respuesta incorrecta», que el cuaderno no planta
- Y esa pregunta no es de la traza: es del juez de ayer. Son complementarios, no sustitutos
- Todo es simulado: esperas programadas y tokens literales. Ninguna llamada a un modelo

Fuente: curso/actividades/semana-4/s4-mar-explicacion.html, líneas 82–92 (el bloque «Encuentra al culpable» y sus 15 min); curso/actividades/_fuentes/traces.jsonl, trazas 4 y 7; _fuentes/s4-mar.py, líneas 141–173 (los cuatro spans de cada traza). Que la tercera traza no exista está verificado sobre el archivo: discrepancia 0.az (ii)

3 · Ejemplos: lo que el código del curso instrumenta, y lo que no

17

Cuatro campos - la traza que el Lab 03 escribe

— Es el andamiaje del Taller 3: su corrección 6 pidió modelo, tokens, latencia y error por paso

— step es un ordinal, no una identidad: no hay trace_id ni span_id ni padre

— Y la última línea tiene el objeto en la mano: response.usage son los tokens ya contados

— El proveedor ya los contó y ya los cobró, y ahí se pierden

```
# curso/labs/Lab-03-Agentes/agent.py, lineas 38-43 y 57-63
@dataclass
class TraceStep:
    step: int
    thought: str | None
    action: dict[str, Any] | None
    observation: Any | None

response = self.client.chat.completions.create(...)
content = response.choices[0].message.content or "{}"
return json.loads(content)   # response.usage se pierde aqui
```

Fuente: curso/labs/Lab-03-Agentes/agent.py, líneas 38–43 (TraceStep completo) y 57–63 (la llamada y la línea que se queda solo con el texto), leídas de primera mano. El archivo final lleva question, answer, trace y status (84–89 y 100–105). curso/talleres/taller-03-agentes.md, Parte 1, «Instrucciones», corrección 6, y «El contrato con el Taller 4» (usage). Discrepancia 0.an, resuelta en el Taller 3 el 2026-09-30; agent.py no cambió, a propósito

Cero de once campos; de nueve piezas, tres a medias

- TraceStep tiene CERO de los once campos mínimos: ni identidad, ni tiempo, ni economía
- Con esa traza, de las cuatro preguntas del principio se responden cero
- El agente corrió perfectamente y la corrida es irrecuperable
- El envoltorio del Lab 04 añade exactamente dos campos: latency_seconds y observed_at
- La Parte 1 pide nueve piezas: cuatro tipos de span, las tres correcciones de la 0.c, el costo y el traza_id. Del andamiaje, ninguna entera
- Y el envoltorio del Lab 04 nunca lee la traza que el Lab 03 escribe: se acoplan por un import

el campo que el andamiaje entregado NO guarda

	campo mínimo	Lab 03	Lab 04
identidad *el trace_id en la traza, los otros dos en cada span*	trace_id	✗ no	✗ no
	span_id	✗ no	✗ no
	padre	✗ no	✗ no
tiempo *en cada span*	inicio	✗ no	✗ no
	duración	✗ no	parcial
economía *solo en los spans de llamada al LLM*	modelo	✗ no	✗ no
	tokens_in	✗ no	✗ no
	tokens_out	✗ no	✗ no
	costo	✗ no	✗ no
cambio *una vez por traza*	prompt_version	✗ no	✗ no
error *en cada span*	error	✗ no	✗ no
		0 de 11	**1 parcial de 11**

los 8 requisitos son estos 11 campos

Lab 03, el agente: agent.py, líneas 38–43 y 66–72 · Lab 04, el envoltorio: instrumented_agent.py, 44–67 · las piezas: Parte 1, puntos 1–4

la Parte 1 (25 %) pide:

4 tipos de span

- corrida — a medias — *sin prompt_version*
- guardrail — a medias — *solo cuando bloquea*
- llm — ✗
- herramienta — a medias — *sin latencia*

3 correcciones de la 0.c

- duración en un finally — ✗
- reloj que incluye los guardrails — ✗
- línea redactada antes de escribirse — ✗
- costo con fila y fecha — ✗
- traza_id en el resultado — ✗

0 enteras · 3 a medias

Fuente: curso/labs/Lab-03-Agentes/agent.py, líneas 38–43 y 66–72; curso/labs/Lab-04-LLMOps/instrumented_agent.py, líneas 44–67; curso/talleres/taller-04-llmops.md, Parte 1, «Instrucciones», puntos 1–4 (las nueve piezas). El barrido de los tres módulos del andamiaje del Lab 04 confirma que no mencionan traces/ ni AGENT_TRACE_DIR; el evaluador nuevo, evaluar_llmops.py, cuenta pasos y errores sobre la traza que devuelve .run(), no sobre el archivo. Los recuentos son [AGREGADO - CÁLCULO DERIVADO]

Dos campos añadidos, cuatro fallas que no fallan y medio acierto

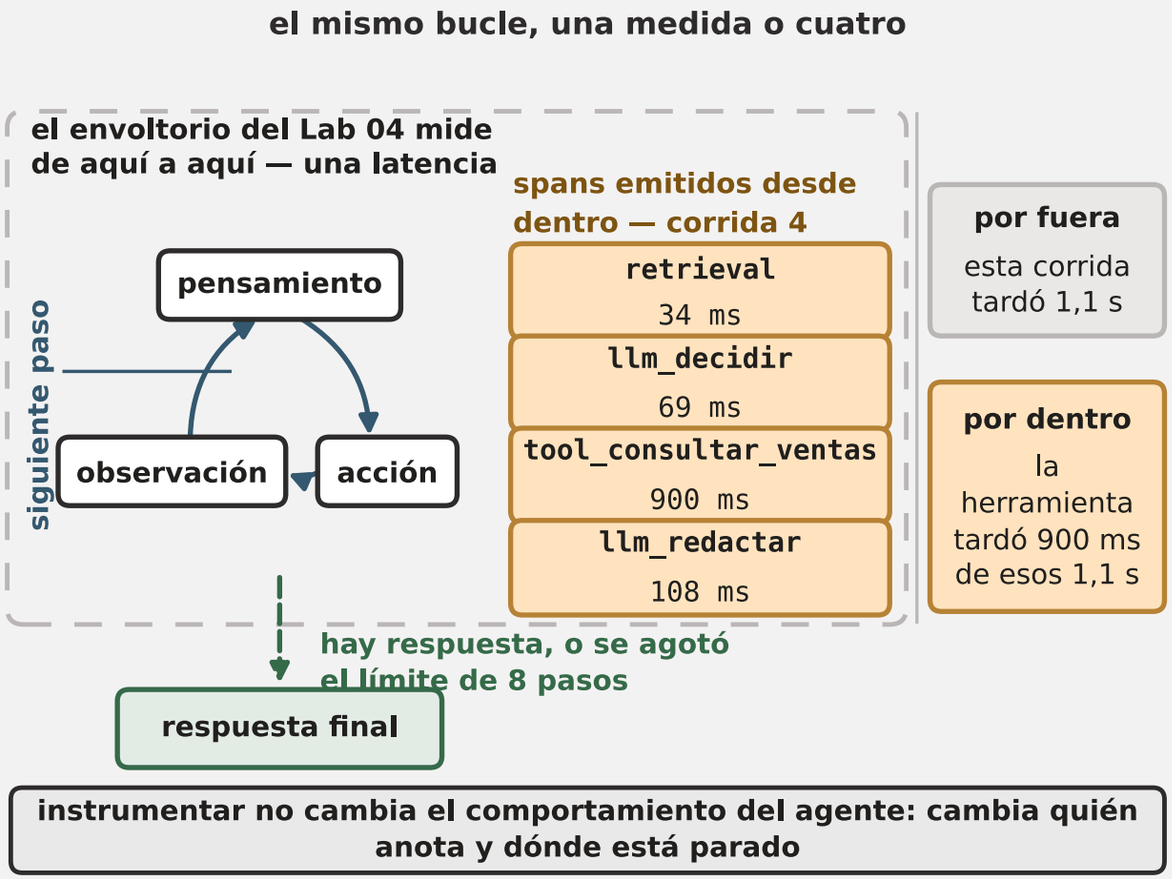
- El reloj arranca en la 45, DESPUÉS del guardrail: uno de 0,4 s queda como latency_seconds=0.0
- Sin try/except en la 50: si el agente lanza, no se escribe ninguna traza
- La rama que bloquea (47) guarda la pregunta ORIGINAL: el correo de un ataque llega a traces.jsonl
- Con USE_LANGFUSE=true, la 59 llama a trace(), que langfuse 4.16.0 no tiene: cae siempre en el except
- Medio acierto: validate_output (51) va ANTES de write_local_trace (58). La entrada bloqueada, no
- Las tres primeras las reproduce python guion.py traza, sin modelo: son la Parte 0.c del viernes 17

```
# curso/labs/Lab-04-LLMOps/instrumented_agent.py, 44-59 (recortado)
ok, cleaned_question = validate_input(question)   # 44
started = time.perf_counter()        # 45 - despues del guardrail
if not ok:
    result = {"question": question, "status": "blocked_input"}  # 47
else:
    result = AnalystAgent().run(cleaned_question)  # 49-50 sin try
    output_ok, safe = validate_output(result["answer"])   # 51
result["latency_seconds"] = round(time.perf_counter() - started, 3)
result["observed_at"] = datetime.now(timezone.utc).isoformat()
write_local_trace(result)       # 58 - despues de validate_output
maybe_langfuse_trace("lab04-agent-run", result)   # 59 -> 36: trace()
```

Fuente: curso/labs/Lab-04-LLMOps/instrumented_agent.py, líneas 29–40 y 43–67, leídas completas (no cambió desde el 2026-09-09, a propósito); curso/labs/Lab-04-LLMOps/guion.py, orden traza (salida del 2026-10-02: correo en el archivo True; 0,41 s reales y latency_seconds=0.0; 2 filas antes y después de la excepción); curso/talleres/taller-04-llmops.md, Parte 0.c; curso/actividades/_fuentes/s4-mar.py, Parte 4, líneas 249–253 (trace no existe en langfuse 4.16.0, comprobado el 2026-10-02); curso/labs/Lab-04-LLMOps/guardrails.py, líneas 23 y 41–44; curso/actividades/semana-4/s4-mar-explicacion.html, líneas 117–118. Los defectos son [AGREGADO - CÁLCULO DERIVADO], contenido.md §6.3

Instrumentar por dentro, no alrededor

- El envoltorio mide UNA latencia de extremo a extremo y no ve ninguna llamada ni herramienta
- El Trazador del cuaderno emite spans desde DENTRO de cada paso del bucle
- Es la diferencia entre «esta corrida tardó 1,1 s» y «la herramienta tardó 900 ms de ese 1,1 s»
- Con la primera no se puede arreglar nada; con la segunda ya sabes qué tocar
- El bucle no cambia: sigue parando por respuesta final o por el límite de pasos



condición de parada leída de Lab-03-Agentes/agent.py, línea 47 (max_steps = 8) · tiempos de _fuentes/traces.jsonl

Fuente: proceso/contexto/glosario.md, entrada «Instrumentación dentro vs. alrededor»; curso/labs/Lab-04-LLMOps/instrumented_agent.py, línea 56 (la única latencia); curso/actividades/_fuentes/s4-mar.py, líneas 144–166 (los cuatro spans emitidos desde dentro de cada paso); curso/labs/Lab-03-Agentes/agent.py, línea 47 (max_steps: int = 8)

El span registra el fallo y lo re-lanza

- El finally garantiza el ms aunque el bloque falle; el except guarda el error
- Y a continuación va raise: instrumentar no es capturar
- La pila fija el padre: la anidación es una estructura, no un campo escrito a mano
- En la sesión 11 la excepción se convertía en observación PARA EL MODELO
- Dos decisiones distintas sobre la misma excepción, y las dos correctas en su sitio
- Es la primera corrección de la Parte 1 del viernes 17: el span de una corrida que falló existe y lleva su error

```
@contextmanager                    # s4-mar.py, 90-105 (recortado)
def span(self, nombre, **attrs):
    s = {"span_id": uuid.uuid4().hex[:8], "inicio": time.time(),
         "padre": self._pila[-1]["span_id"] if self._pila else None}
    self._pila.append(s)
    try:
        yield s
    except Exception as e:
        s["error"] = str(e); raise   # registra el fallo y RE-LANZA
    finally:
        s["ms"] = round((time.time() - s["inicio"]) * 1000, 2)
        self._pila.pop(); self._actual["spans"].append(s)
```

Fuente: curso/actividades/_fuentes/s4-mar.py, líneas 90–105 (el context manager completo; se recortan los campos nombre y attrs del diccionario y el error = None tras el yield, por espacio), 78 («un Langfuse de bolsillo», comentario del autor) y 73–75 (la duración en el finally, dicha en el enunciado del 1.1 desde el 2026-09-25); curso/talleres/taller-04-llmops.md, Parte 1, «Instrucciones», punto 2. El contraste con el except de la sesión 11 es proceso/contexto/glosario.md, entrada «Tool error as observation»

Instrumenta y construye tu mini-dashboard - 50 minutos

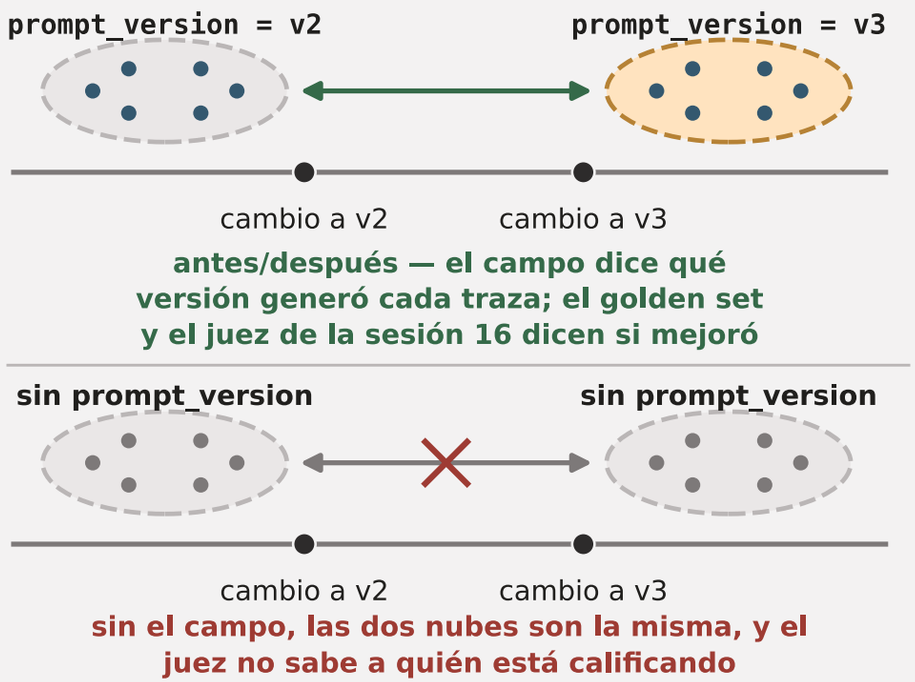
- 1 · Trazador: nueva_traza, span, cerrar y guardar a JSONL
- 2 · Instrumenta el agente simulado y genera 10 corridas → traces.jsonl
- 3 · Mini-dashboard: costo por corrida, latencia con p50 y p95, latencia por tipo de span
- 4 · Diagnóstico: imprime los spans de la corrida más cara y de la más lenta
- Corre entero sin ninguna clave de API y sin gastar un centavo
- El 1.1 pide también la instancia, trazador = Trazador(): las Partes 2 y 3 la usan por ese nombre

Fuente: curso/actividades/semana-4/s4-mar-explicacion.html, líneas 94–104 (los cuatro pasos y los 50 min); curso/actividades/_fuentes/s4-mar.py, líneas 60–71 (el enunciado del ejercicio 1.1, que pide la instancia desde el 2026-09-25), 33–55 (la tabla semestral se busca subiendo directorios y los precios se leen de ella) y 122 (la instancia, en la solución). El NameError de la versión de estudiante (0.ay) quedó corregido ese día, en el enunciado

prompt_version - por qué la cuarta familia existe

- «Los prompts son código y pueden cambiar independientemente del modelo»
- «Sin la versión en la traza, el antes/después es imposible de reconstruir»
- Hoy se practica en dos líneas: prompt_version="v3" al abrir cada traza
- Y «v3» es constante en las diez corridas: nunca hay una v2 con la que comparar
- El antes/después se mide con lo de ayer: golden set y juez. La versión dice **qué** cambió; la evaluación, **si mejoró**
- El Taller 4 lo evalúa en la Parte 2.c (10 %): la mejora va como versión nueva, con changelog y `prompt_version` en cada span

el antes/después necesita un campo, no una intención



el ciclo de vida, sesión 18

- registro
- changelog
- evaluación por versión
- rollback

en la corrida del curso prompt_version vale v3 en las 10 trazas: no hay una v2 real, así que hoy se instala el campo y no se ejercita la comparación

las notas dan dos sitios para el artefacto: archivos de prompt versionados en git, o gestión en una plataforma

Fuente: fuentes/notas-teoricas/s4-llmops-y-finetuning.md §1, línea 12 y §3.4, líneas 152–158 (las dos opciones y el porqué del rollback, literales); curso/actividades/semana-4/s4-mar-explicacion.html, líneas 114–115; _fuentes/s4-mar.py, línea 142; _fuentes/traces.jsonl (v3 en las diez trazas, verificado); curso/talleres/taller-04-llmops.md, Parte 2.c y «Qué cambió el 2026-10-02» (hasta el 2026-10-01 el enunciado no nombraba versión, versionado, changelog ni rollback — contenido.md §9.1); slides_v2/sesion-16/sesion-16.qmd, diapositiva de cierre (el golden set y el juez como instrumento de medida). El reparto 17/18 es [POR CONFIRMAR]: discrepancia 0.av, resuelta el 2026-10-02 bajo el supuesto 0.by (viii), que el profesor no ha confirmado

4 · Langfuse: los mismos objetos, con nombre propio

17

Por qué Langfuse, y en qué versión

- Código abierto (MIT) y **autoalojable con Docker**: las trazas de su agente no salen de su máquina si no quieren
- Construido sobre OpenTelemetry: traza y span son los del estándar del tracing distribuido, no un formato inventado
- `pip install langfuse==4.17.0`: pide Python ≥ 3.10, el curso usa 3.11+, y convive con openai 3.22.1 del Lab 03
- Servidor: Langfuse 4.50.0 en Docker para la demo de hoy; en casa, el mismo Docker o Langfuse Cloud, plan Hobby gratis
- **Fijen la versión**: el SDK cambió de API entre la v3 y la v4, y el envoltorio del Lab 04 todavía llama a la v3
- Desde hoy el Taller 4 lo exige en la Parte 1. Lo que se califica sigue siendo un archivo: el JSONL que `exportar_trazas.py` del Lab 04 baja de Langfuse

Fuente: fuentes/web/fuentes-web.md, bloques «Consultadas el 2026-10-06» (PyPI, SDK de Python, autoalojamiento y versiones del servidor) y «Consultadas el 2026-10-02» (precios: plan Hobby); curso/labs/Lab-04-LLMOps/instrumented_agent.py, línea 59 (la llamada de la v3); curso/silabo/silabo.md, línea 125 (Python 3.11+); curso/labs/Lab-04-LLMOps/exportar_trazas.py y curso/talleres/taller-04-llmops.md, «Qué cambió el 2026-10-06». La compatibilidad con openai 3.22.1 se comprobó instalando curso/labs/Lab-03-Agentes/requirements.txt junto a langfuse 4.17.0 el 2026-10-06

Cómo se llama cada objeto en Langfuse

- **trace** = la traza, una corrida entera → la función raíz decorada con `@observe`
- **observation** = el span, un paso → cada `@observe` o cada `with` abierto dentro de otro
- **generation** = el span de una llamada al LLM, con modelo y tokens → `as_type="generation"`, o el cliente `langfuse.openai`
- **tool**, **retriever**, **agent**, **guardrail** = spans con tipo → `as_type="tool"`, `as_type="retriever"`…
- **session** = varias trazas del mismo hilo o del mismo grupo → `propagate_attributes(session_id=...)`
- **prompt**, con versión y etiqueta = su `prompt_version`, ahora gestionado → `get_prompt()` y `langfuse_prompt=`
- **score** = una verificación, o el veredicto del juez de ayer, pegado a la traza → `score_current_trace()`

Fuente: fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-02» (Langfuse, modelo de datos: traza, observación, sesión) y «Consultadas el 2026-10-06» (tipos de observation, `propagate_attributes`, gestión de prompts, scores; las firmas, leídas de langfuse 4.17.0 instalado). La correspondencia con el Trazador del curso es [AGREGADO - formulación]

Paso a paso 1 de 4 · Levantar Langfuse con Docker

- Necesitan Docker Desktop corriendo. La primera vez se bajan unos 4 GB de imágenes
- Opción A, con el repositorio del curso: el script de la demo hace todo, claves incluidas
- Opción B, en cualquier máquina: el compose oficial en la versión de hoy, y la cuenta se crea en la interfaz local
- Si choca un puerto —un Postgres propio en el 5432—, usen la opción A, que quita los puertos internos

```
# Opción A — repositorio del curso
cd curso/actividades/semana-4/demo-langfuse
./levantar_langfuse.sh                # proyecto, claves y usuario ya creados

# Opción B — compose oficial, versión fijada
git clone --depth 1 --branch v4.50.0 https://github.com/langfuse/langfuse.git
cd langfuse && docker compose up -d
# http://localhost:3000 → Sign up → New project → Settings → API Keys → Create
```

Fuente: curso/actividades/semana-4/demo-langfuse/levantar_langfuse.sh y README.md; fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (Docker Compose, inicialización headless, servidor 4.50.0). La opción A se ejecutó el 2026-10-06 en macOS con Docker 29.8.1

Paso a paso 2 de 4 · El SDK y las claves

- Un entorno virtual con Python 3.11+ y tres paquetes con su versión fijada
- Las claves del proyecto y la URL del servidor van en `.env`; el `.env`, en `.gitignore`
- El modelo, igual que en el Lab 03: la H200 con la VPN, u Ollama local sin ella
- Antes de seguir, comprueben la conexión: `auth_check()` tiene que imprimir `True`

```
python -m venv .venv && source .venv/bin/activate
pip install langfuse==4.17.0 openai==3.22.1 python-dotenv==1.2.3

cat >> .env <<'EOF'
LANGFUSE_PUBLIC_KEY=pk-lf-...            # Settings → API Keys
LANGFUSE_SECRET_KEY=sk-lf-...
LANGFUSE_BASE_URL=http://localhost:3000
OPENAI_BASE_URL=http://<host de la H200>:12555/v1   # o http://localhost:11434/v1
OPENAI_API_KEY=<la del laboratorio>                 # con Ollama: ollama
EOF

python -c "from dotenv import load_dotenv; load_dotenv(); \
from langfuse import get_client; print(get_client().auth_check())"   # True
```

Fuente: curso/actividades/semana-4/demo-langfuse/requirements.txt y README.md; curso/labs/Lab-04-LLMOps/.env.example (las variables del modelo); fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (variables `LANGFUSE_*`, `auth_check`); curso/politica-credenciales.md. Ejecutado el 2026-10-06: `auth_check()` devolvió True contra el servidor local

Paso a paso 3 de 4 · Un ejemplo completo, para copiar

— Veinte líneas: una traza (`resumir`), una generation (`llm_resumir`) y una herramienta (`contar_palabras`)

— El modelo no se escribe a mano: se le pregunta al servidor cuál sirve

— `flush()` al final: sin esa línea, un script tan corto termina antes de enviar nada

```
from dotenv import load_dotenv; load_dotenv()      # LANGFUSE_* y OPENAI_* del .env
from langfuse import observe, get_client
from langfuse.openai import OpenAI                   # el cliente de OpenAI, con trazas

llm = OpenAI()
MODELO = llm.models.list().data[0].id                # el que sirve el servidor

@observe(name="contar_palabras", as_type="tool")
def contar_palabras(texto):
    return len(texto.split())

@observe(name="resumir")                             # la raíz: una traza por llamada
def resumir(texto):
    r = llm.chat.completions.create(model=MODELO, name="llm_resumir",
        messages=[{"role": "user", "content": f"Resume en una frase: {texto}"}])
    resumen = r.choices[0].message.content
    return resumen, contar_palabras(resumen)

print(resumir("Langfuse guarda trazas y observations de una aplicación con LLM."))
get_client().flush()                                 # enviar antes de terminar
```

Fuente: curso/actividades/semana-4/demo-langfuse/hola_langfuse.py, completo. Ejecutado el 2026-10-06 con langfuse 4.17.0 contra Langfuse 4.50.0 y el vLLM de la H200

Paso a paso 4 de 4 · Correrlo y verlo

- En *Tracing*, la traza `resumir`: clic, y a la izquierda el árbol, a la derecha la línea de tiempo
- `llm_resumir` es la generation: modelo, tokens de entrada, de salida y de razonamiento, y su latencia
- `contar_palabras` es la tool: su entrada y su salida. Si hubiera lanzado, saldría en rojo, con el mensaje
- Córranlo tres veces y vuelvan a la lista: cada corrida es una fila con su latencia y sus tokens

```
$ python hola_langfuse.py
('Langfuse registra y almacena las trazas y observaciones…', 22)

http://localhost:3000 → Tracing → resumir
trace       resumir            2 141 ms
├─ generation  llm_resumir     2 139 ms   entrada 31 · salida 34 · razonamiento 272
└─ tool        contar_palabras    < 1 ms   salida 22
```

Fuente: corrida de curso/actividades/semana-4/demo-langfuse/hola_langfuse.py el 2026-10-06; las cifras se leyeron de Langfuse 4.50.0 por GET /api/public/v2/observations. El dibujo es [AGREGADO - redibujo de la vista de traza]

Ejemplo 1 · Dos líneas y la traza sale sola

- `@observe` en la función raíz: su llamada **es** la traza; cada función decorada dentro es una observation hija
- `langfuse.openai` es el mismo cliente de OpenAI: registra modelo, tokens —también los de razonamiento—, latencia y error
- Con la H200 funciona igual: basta su `base_url`, que es compatible con OpenAI

```
from langfuse import observe
from langfuse.openai import OpenAI      # en vez de: from openai import OpenAI

llm = OpenAI()                          # OPENAI_BASE_URL apunta a la H200

@observe(name="buscar_contexto", as_type="retriever")
def buscar(pregunta): ...

@observe(name="responder")              # la raíz: una traza por llamada
def responder(pregunta):
    contexto = buscar(pregunta)         # observation hija
    r = llm.chat.completions.create(model=MODELO, name="redactar", messages=[...])
    return r.choices[0].message.content  # generation hija, con sus tokens
```

Fuente: curso/actividades/semana-4/demo-langfuse/demo.py, funciones buscar y responder (recortadas); fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (integración con OpenAI e instrumentación). Ejecutado el 2026-10-06 con langfuse 4.17.0 y openai 3.22.1 contra el vLLM de la H200

Ejemplo 2 · Por dentro y a mano: modelo, tokens y error

- Sin envoltorio —otro proveedor, una llamada cruda— la generation se abre a mano y se le dan modelo y tokens
- `usage_details` son los tokens que el proveedor ya contó: el `response.usage` que el Lab 03 tiraba
- Si el bloque lanza, el span queda con estado de error y la excepción **sigue**: el `finally` del Trazador, hecho por el SDK

```
from langfuse import get_client
lf = get_client()                                  # lee LANGFUSE_* del .env

with lf.start_as_current_observation(name="corrida", as_type="agent"):
    with lf.start_as_current_observation(
            name="llm_decidir", as_type="generation", model=MODELO) as gen:
        r = cliente.chat.completions.create(...)
        gen.update(output=r.choices[0].message.content,
                   usage_details={"input": r.usage.prompt_tokens,
                                  "output": r.usage.completion_tokens})
    with lf.start_as_current_observation(name="consultar_sql", as_type="tool"):
        ejecutar(sql)                              # si lanza: error en el span y re-lanza
```

Fuente: fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (`start_as_current_observation`, `usage_details`; firmas leídas de langfuse 4.17.0); curso/labs/Lab-03-Agentes/agent.py, líneas 57–63 (la respuesta de la que solo se guarda el texto). Que el error quede en el span y la excepción se propague se comprobó el 2026-10-06 con un exportador en memoria [AGREGADO - medición propia]

Ejemplo 3 · El prompt es un artefacto: versión, etiqueta y rollback

- `create_prompt` con un nombre que ya existe crea la **versión siguiente**, con su mensaje de cambio: un changelog
- La aplicación pide una **etiqueta**, no un número: desplegar es mover `production`; revertir, moverla de vuelta
- `langfuse_prompt=` liga cada generation a su versión: latencia, tokens y scores **por versión**

```
lf.create_prompt(name="asistente-calendario", prompt=V2, labels=["candidata"],
                 commit_message="v2: solo el contexto; «No lo sé» si no está")

prompt = lf.get_prompt("asistente-calendario", label="production",
                       fallback=V1)           # si Langfuse cae, la app sigue
r = llm.chat.completions.create(
        model=MODELO, langfuse_prompt=prompt,   # esta generation es de esa versión
        messages=[{"role": "system", "content": prompt.compile(contexto=ctx)}, ...])

lf.update_prompt(name="asistente-calendario", version=2,
                 new_labels=["production"])   # desplegar; con version=1, rollback
```

Fuente: curso/actividades/semana-4/demo-langfuse/demo.py, funciones registrar_prompts, trazar y promover; fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (gestión de prompts y ligar prompts a trazas). Ejecutado el 2026-10-06 contra Langfuse 4.50.0 autoalojado: tras promover, la etiqueta production apunta a la versión 2

Ejemplo 4 · Antes de enviar: máscara, sesión y flush

- La máscara corre **en su máquina**, antes de enviar: el correo no llega nunca al servidor
- `@observe` entrega los argumentos como **tupla**: una máscara que solo recorre listas dejó pasar el correo, sin avisar
- `propagate_attributes` pone sesión y etiquetas a todas las observations de dentro; `flush()` antes de que el script termine

```
CORREO = re.compile(r"[\w.+-]+@[\w-]+\.[\w.-]+")

def enmascarar(*, data, **_):
    if isinstance(data, str):  return CORREO.sub("<CORREO>", data)
    if isinstance(data, dict): return {k: enmascarar(data=v) for k, v in data.items()}
    if isinstance(data, (list, tuple)):              # la tupla es la que se olvida
        return [enmascarar(data=v) for v in data]
    return data

Langfuse(mask=enmascarar)                # una vez; get_client() la reutiliza
with propagate_attributes(session_id="grupo-07", tags=["taller-04"]):
    responder("Soy ana.perez@usfq.edu.ec: ¿cuándo es mi examen?")
get_client().flush()
```

Fuente: curso/actividades/semana-4/demo-langfuse/demo.py, función enmascarar y paso falla; fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-06» (máscara y «Lo que no dicen»). La fuga por la tupla se reprodujo el 2026-10-06 con langfuse 4.17.0 [AGREGADO - medición propia]

Cómo se ve una traza: el árbol y la línea de tiempo

- La vista de una traza en Langfuse es este árbol con su línea de tiempo: dónde se fue cada milisegundo y cada token
- Con la v1, la pregunta fuera de alcance gastó **140 tokens de entrada**, el doble que las demás: el buscador mandó la tabla entera
- Y su score `contiene_dato` es 0: la traza dice **qué** pasó; el score, **si estuvo bien**
- La lista de trazas filtra por sesión, etiqueta y versión; la página del prompt pone las versiones lado a lado

```
trace  responder   «¿Cuál es la capital de Mongolia?»   sesión demo-production
│ 0 ms                                                     1 127 ms
├─ retriever   buscar_contexto   |                          < 1 ms
└─ generation  redactar          ═══════════════════════  1 126 ms
                prompt asistente-calendario v1
                tokens: entrada 140 · salida 57 · razonamiento 144 · total 341
score  contiene_dato = 0     esperaba «No lo sé»; respondió la capital
```

Fuente: corrida de curso/actividades/semana-4/demo-langfuse/demo.py, paso trazas con la etiqueta production, el 2026-10-06; las cifras se leyeron de Langfuse 4.50.0 por GET /api/public/v2/observations (latencias, usage_details y prompt ligado). El dibujo es [AGREGADO - redibujo de la vista de traza]

Demo en vivo · Langfuse en Docker, 10 minutos

- `./levantar_langfuse.sh`: Langfuse en localhost:3000, con proyecto, claves y usuario creados sin pasar por la interfaz
- **1** · `python demo.py sin-trazas`: cinco respuestas, y ninguna de las cuatro preguntas del principio tiene respuesta
- **2** · `prompts` y `trazas`: la v1, trazada. Abrimos la traza de Mongolia y su línea de tiempo
- **3** · `trazas --label candidata`: la v2 responde «No lo sé». Comparamos las dos versiones en la página del prompt
- **4** · `promover 2`: production cambia sin redesplegar; `promover 1` es el rollback
- **5** · `falla`: la herramienta en ERROR, la respuesta con HTTP 200, y el correo guardado como `<CORREO>`

Fuente: curso/actividades/semana-4/demo-langfuse/README.md (el guion) y levantar_langfuse.sh; la demo es más pequeña que el laboratorio a propósito: un buscador de cuatro filas y una llamada al modelo, sin herramientas SQL

Recomendaciones de uso

- Fijen la versión del SDK —`langfuse==4.17.0`— en su requirements.txt y declárenla en el informe
- Las claves van en `.env`, que está en `.gitignore`; nunca en el código ni en el repositorio
- Un solo cliente: `Langfuse(...)` una vez, con su máscara, y `get_client()` en todo lo demás
- Liguen cada generation a su prompt: sin eso, las métricas por versión no existen
- Enmascaren antes de enviar, y pruébenlo con un dato real mirando lo que quedó guardado
- `flush()` al final de cada script; la traza exportada a JSONL es lo que se entrega
- Docker Compose sirve para clase y pruebas: le faltan alta disponibilidad, escalado y copias de respaldo

Fuente: fuentes/web/fuentes-web.md, bloques «Consultadas el 2026-10-06» (SDK, máscara, gestión de prompts, autoalojamiento) y «Consultadas el 2026-10-02» (Docker Compose); curso/politica-credenciales.md (las claves). La lista es [AGREGADO - síntesis de la demo y de la documentación]

Las alternativas: trazas propias y otras plataformas

- Langfuse no es la única ruta: LangSmith, de la sesión 14, o unas trazas propias en JSONL resuelven lo mismo con otros costos
- Setup: «~30 líneas de Python, cero dependencias» frente a «SDK + cuenta o self-host»
- Consulta: «pandas / jq sobre el archivo» frente a «UI: búsqueda, filtros, gráficos»
- Colaboración: «compartir archivos» frente a «el equipo viendo el mismo dashboard»
- Autoalojar Langfuse con Docker Compose sirve para probar: le faltan alta disponibilidad, escalado y copias de respaldo (consultado el 2026-10-02)
- El Taller 4 exige Langfuse desde el 2026-10-06: el Trazador queda como modelo mental, y el JSONL exportado, como lo que se entrega

dos maneras de operar, no dos productos

criterio	trazas locales JSONL — el baseline del Lab 04	plataforma gestionada — obligatoria en el Taller 4
Setup	~30 líneas de Python, cero dependencias	SDK + cuenta o self-host
Consulta	pandas / jq sobre el archivo	UI: búsqueda, filtros, gráficos de costo y latencia
Colaboración	compartir archivos	equipo viendo el mismo dashboard; anotaciones
Autoalojar	no aplica: es un archivo	Langfuse con Docker Compose: sirve para probar — sin alta disponibilidad, escalado ni copias de respaldo *consultado el 2026-10-02*

desde el 2026-10-06 la Parte 1 (25 %) exige Langfuse — y lo que se califica es el JSONL que se exporta de él

filas 1–3, s4-mar-explicacion.html, líneas 69–80; fila 4, fuentes/web/fuentes-web.md, «Consultadas el 2026-10-02»

Fuente: curso/talleres/taller-04-llmops.md, Parte 1, «Instrucciones» (hasta el 2026-10-05 las tres rutas «valen lo mismo»; desde el 2026-10-06 Langfuse es obligatorio y la evidencia es el JSONL exportado, sin captura de ningún panel); curso/actividades/semana-4/s4-mar-explicacion.html, líneas 69–80 (la tabla, literal; compara trazas locales y plataforma gestionada, no dos productos) y línea 32 (el objetivo: «qué agrega Langfuse sobre las trazas locales»); fuentes/web/fuentes-web.md, bloque «Consultadas el 2026-10-02», Langfuse, autoalojamiento; curso/labs/Lab-04-LLMOps/resumir_trazas.py, docstring; curso/silabo/silabo.md, línea 132

¿Qué falta todavía para decir que A es mejor que B?

Hasta el 30 de septiembre esta tabla no se podía llenar: de LangSmith solo había el nombre, y de ninguna de las dos una página fechada

Desde el 1 y el 2 de octubre hay documentación fechada de las dos, y versión: langfuse 4.16.0, langsmith 0.14.2

Las dos llaman traza a una corrida; cada paso es una *observation* en Langfuse y un *run* en LangSmith

Gratis: Langfuse Hobby, 50k unidades al mes; LangSmith Developer, hasta 5k trazas base al mes

Con todo eso, ¿se puede decir cuál conviene a su agente? Si no, ¿qué falta?

Y la gemela: ¿cuántas de esas condiciones cumple su comparación v1 → v2 del viernes 17?

la tabla que el objetivo pide, con lo que hay fechado

documentación consultada el 2026-10-01 y el 2026-10-02

	Langfuse	LangSmith
una corrida	traza	traza
cada paso	*observation*	*run*
versión	`4.16.0`	`0.14.2`
gratis	Hobby: 50k unidades al mes	Developer: hasta 5k trazas base al mes
¿cuál conviene a su agente?	?	

qué cambió

LangSmith
hasta el 30 de septiembre: solo el nombre
desde octubre: conceptos y planes consultados con fecha

Langfuse
hasta el 30 de septiembre: tres formas de instrumentar que no coincidían
desde octubre: el cuaderno en el SDK v4, comprobado con 4.16.0, y el envoltorio del Lab 04 con un trace() que esa versión no tiene

fuentes-web.md
hasta el 30 de septiembre: ninguna página fechada de las dos
desde octubre: un bloque «Consultadas el …» por día

es el criterio 7 de la rúbrica de este curso; y la misma vara se aplica a su comparación v1 → v2 del sábado

Fuente: fuentes/web/fuentes-web.md, bloques «Consultadas el 2026-10-02» (Langfuse: modelo de datos, SDK de Python —4.16.0 en PyPI, 2026-09-30— y precios; LangSmith: conceptos) y «Consultadas el 2026-10-01» (LangSmith: planes; langsmith 0.14.2, leída de los metadatos instalados con proceso/scripts/verificar_tutorial_s14.py); curso/actividades/_fuentes/s4-mar.py, Parte 4, líneas 242–276 (el SDK v4, comprobado con 4.16.0 el 2026-10-02); curso/labs/Lab-04-LLMOps/instrumented_agent.py, líneas 29–40; curso/silabo/silabo.md, líneas 132 y 219. Discrepancias 0.ar y 0.aw (resuelta en parte el 2026-10-02)

Mañana el ciclo de vida; el viernes 17, la Parte 1

- Instrumentar es meterse dentro del bucle: por fuera solo hay una latencia
- response.usage es la línea que hay que no descartar: sin tokens no hay costo
- Publiquen percentiles y conserven el span: la media esconde una anomalía y fabrica otra
- El viernes 17: la Parte 0.c reproduce las fallas del envoltorio y la Parte 1 pide los spans **instrumentados con Langfuse** y exportados a JSONL
- Con Langfuse: `@observe` en la raíz, el prompt ligado a cada generation, la máscara probada y `flush()` al final
- Taller 4 — 25 % de la nota · viernes 17 de octubre · lo de hoy: Parte 0.c (3 %) y Parte 1 (25 %)
- Mañana el prompt es un artefacto con changelog y rollback. Hoy dejen puesto el campo: la Parte 2.c lo califica